



Savitribai Phule Pune University

**T.Y. B.Sc (Cyber and Digital
Science) Semester-VI**

**CDS-366: Lab Course on CDS-363 -
Web Science**

Name: _____

College Name: _____

Roll No.: _____ Division: _____

Academic Year : _____

CERTIFICATE

This is to certify that Mr. /Ms. _____

Seat Number _____ of T.Y.B.Sc. (Cyber and Digital Science) Sem-VI

has successfully completed Lab course CDS-366: Lab course on Web Science in the

Academic Year _____.

He/ She has scored _____ mark out of 10 (For Lab Book).

Subject Teacher

H.O.D./Coordinator

Internal Examiner

External Examiner

Introduction

1. About the Workbook:

This workbook is intended to be used by T.Y.B.Sc.(Cyber and Digital Science) Semester-VI students for Web Science Assignments. This workbook is designed by considering all the practical concepts / topics mentioned in syllabus.

2. The objectives of this Workbook are:

- 1) Defining the scope of the course.
- 2) To bring the uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
- 3) To have continuous assessment of the course and students.
- 4) Providing ready reference for the students during practical implementation.
- 5) Provide more options to students so that they can have good practice before facing the examination.
- 6) Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. How to use this Workbook:

The workbook contains eight assignments

It is mandatory for students to complete all the assignments in given slot.

4. Instructions to the students:

Please read the following instructions carefully and follow them.

- Students are expected to carry this workbook every time they come to the lab for practical.
- Students should prepare for the assignment by reading the relevant material which is mentioned in ready reference.
- Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However, student should spend additional hours in Lab and at home to cover all workbook assignments if needed.
- Students will be assessed for each assignment on a scale from 0 to 5.

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

5. Instruction to the Instructors:

Make sure that students should follow above instructions.

- Explain the assignment and related concepts using white board if required or by demonstrating the software.
- Give specific input to fill the blanks in queries which can vary from student to student.
- Evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.

The value should also be entered on assignment completion page of the respective Lab course.

6. Instructions to the Lab administrator:

- You have to ensure appropriate hardware and software is made available to each student.

Editors:

Mr. Satyavan Kunjir

Dr. D. Y. Patil ACS College, Pimpri, Pune

Mrs. Aradhya Desai

Dr. D. Y. Patil ACS College, Pimpri, Pune

Ms. Shraddha Khorgade

Dr. D. Y. Patil ACS College, Pimpri, Pune

Reviewed By:

Dr. Ranjit D. Patil

Dr. D. Y. Patil ACS College, Pimpri, Pune

Dr. Sujata Patil

Dr. D. Y. Patil ACS College, Pimpri, Pune

Assignment Index

Sr. No.	Assignment Name	Page Number
1	Use of HTML	7
2	Introduction to PHP	15
3	Introduction to XML	19
4	Databases	23
5	Authorization and Authentication	28
6	Web Services	32
7	Introduction OWASP	36
8	Introduction to Web Security audit tools	38

Signature of Instructor:

Assignment Completion Sheet

Sr. No.	Assignment Name	Marks (out of 5)	Teacher's Sign
1	Use of HTML		
2	Introduction to PHP		
3	Introduction to XML		
4	Databases (2 Slots)		
5	Authorization and Authentication		
6	Web Services		
7	Introduction OWASP		
8	Introduction to Web Security audit tools		
Total (Out of 35)			
Total (Out of 5)			

Signature of Instruction

Assignment1

Use of HTML

Introduction to HTML

HTML stands for Hypertext Mark-up Language. It is the standard mark-up language for creating web pages and web applications. HTML is the World Wide Web's core mark-up language. Without HTML, the WWW will not exist. It is the most widely used language to write web pages. In HTML Hypertext simply means "Text within Text". A text has a link within it, is a hypertext.

Features of HTML

- HTML is a very easy and simple language. It can be easily understood and modified.
- It is a markup language, so it provides a flexible way to design web pages along with the text.
- It is platform-independent because it can be displayed on any platform like Windows, Linux and Macintosh etc.
- It facilitates the programmer to add Graphics, Videos, and Sound to the web pages which makes it more attractive and interactive.
- It is very easy to make effective presentation with HTML because it has a lot of formatting tags.
- It supports scripting languages to support dynamic web applications (e.g., forms that react to user input).

Advantages of HTML

- HTML is highly flexible and user-friendly.
- It is an open technology that supports almost all the web browsers and platforms like Windows, Macintosh, etc.
- HTML is efficient and reliable. We can create the Web page in order to advertise and promote products and services.
- It is easily understandable and does not require long time training.
- It provides search engine compatible to the Web sites.

HTML Page Structure

- A web page is also known as HTML page or HTML document. HTML files are text files featuring semantically tagged elements.
- HTML filenames are suffixed with ".htm or .html" file extensions.
- A web page is marked by an opening <html>tag and a closing </html> tag and is divided into the

Following three major sections

- 1. Comment Section (optional):** This section contains comments about the web page. It is important to include comments that tell us what is going on in the web page.
- 2. Head Section (optional):** The head section is defined with a starting <head> tag and closing </head> tag. This section usually contains a title for the Web page.
- 3. Body Section:** The body section comes after the head section. The body section contains the entire information about the web page and its behavior. A web page outline containing these three sections and the opening and closing HTML tags.

Creating HTML Document

- Creating an HTML document is very easy and simple. To begin coding HTML we need only two things i.e., a text editor and a web browser. Notepad is the most basic of text editor and we will probably code a fair amount of HTML with it. Follow the following steps for creating a web page.
- **Step 1:** Open Notepad or another text editor and type the following code given in following example.
- **Step 2:** Then, in the File menu, choose Save As to save the program. Name the file Program_1.htm or Program_1.html then Click Save.
- **Step 3:** Then go to the folder in which we save the HTML file. Then Double Click on Program_1 File which displays the output.

Example for creating a html document

```
<!DOCTYPE html>
<!--Example for creating a html document-->
<html>
<head>
<title> This is your title </title>
</head>
<body>
<h1> This is your header </h1>
<p> This is your paragraph</p>
</body>
</html>
```

In above Example

- **<!DOCTYPE html>** This tag specifies the language you will write on the page. In this case, the language is HTML5.
- **<html>** This tag signals that from here onwards we are going to write in HTML code
- **<head>** This is where all the metadata for the page goes-stuff mostly meant for search engines and other computer programs.
- **<body>** This is where the content of the page goes.
- Inside the **<head>** tag, there is one tag that is always included; **<title>**.
- **<title>** This is where we insert the page name as it will appear at the top of the browser window.

Working with elements of HTML

- HTML elements are the building blocks of HTML pages. HTML uses elements to specify a document's structure, to provide information and to format its contents.
- HTML elements are defined or represented by using HTML tags. A tag pair (start tag and end tag) and any data it contains are referred to as an element.
- An HTML element consists of a start tag **<element name>**, its contents and an end tag **</element name>**. Element and tag can be used interchangeably. The basic syntax of an HTML element is given below:
<element name> element content </element name>

- **For example:** **<h1>Heading1</h1>**, where **<h1>** is the open/start tag and **</h1>** is an end tag. The information contained within an element is opening and closing tags is referred to as its contents.

- However, the HTML elements with no content are called empty elements. Empty elements do not have an end tag, such as the
 element (which indicates a line break).
- HTML elements can be nested (elements can contain elements). Attributes can provide additional information about the HTML elements. For example, consider the body element <body> with an added bgcolor attribute, we can tell the browser that the background color of the page should be red, like this <body bgcolor="red">.

The basic elements of an HTML page are given below

1. A text header elements, denoted using the <h1>, <h2>, <h3>, <h4>, <h5>, <h6> tags.
2. A paragraph elements, denoted using the <p> tag.
3. A horizontal ruler elements, denoted using the <hr> tag.
4. A link elements, denoted using the <a> (anchor) tag.
5. A list elements, denoted using the (unordered list), (ordered list) and (list items) tags.
6. An image elements, denoted using the tag.
7. A divider elements, denoted using the <div> tag.

HTML Tags

- HTML documents are simply a text file made up of HTML elements and these elements are defined using HTML tags.
- The information within an element's opening and closing tags is its content. A tag contains three parts i.e. element (identification of tag), attribute and value.
- Syntax of Tag: <tag_name>content.... </tag_name>
- HTML tags tells, the browser which elements to present and how to present them. Where the element appears is determined by the order in which the tags appear. HTML tags sometimes require additional information to be supplied to them.
- The additional information supplied to an HTML tag is known as attributes of a tag.
- Attributes are written immediately following the tag, separated by a space like Welcome .
- Multiple attributes can be associated with a tag, also separated by a space like Welcome . The face and size are multiple attributes of the tag.

Basic HTML Tags

Tag	Example	Description
Headings <h1>	<h1>This is a h1 heading</h1> • Headings are defined with the <h1> to <h6> tags	• <h1> defines the largest heading and <h6> defines the smallest heading
Paragraphs <p>	This is a paragraph This is another paragraph	• Paragraphs are defined with the <p> tag. • HTML automatically adds an extra blank line before and after a paragraph.

Line Breaks 	This is a para graph with line breaks	• The tag is used when you want to end a line, but don't want to start a new paragraph.
Horizontal Line <hr>	It is a course designed for S.Y.B.Sc. (Cyber and Digital Science) <hr> HTML	• To separate your paragraphs with a horizontal line instead of a simple HTML paragraph break, use the <hr> element
Scrolling <marquee>	<marquee> Example of marquee</marquee>	• The Marquee HTML tag is a non-standard HTML element which is used to scroll an image or text horizontally or vertically

Text Formatting Tags

Tag	Example	Description
	This text will appear bold	Used to represent text in bold
<u></u>	This is <u> underlined text </u>	To make text appear underlined
<i></i>	This is <i> italic text </i>	To make text appear italic
<center> </center>	<center> Text is centered text</center>	Centers enclosed text
<big></big>	<big> This will appear one size big</big>	Sets the type one font larger than the surrounding text
	This will appear one size small	Sets the type one font smaller than the surrounding text

This assignment involves creating a webpage using HTML with a specific layout and functionality. Below is a detailed explanation of the tasks and their implementation.

HTML attributes

- All HTML elements can have attributes.
- Attributes provide additional information about elements.
- Attributes are always specified in the start tag.
- Attributes usually come in name/value pairs like: name="value"

HTML Tags with attributes

Tag	Example	Description
HTML color	<pre><body bgcolor="yellow"> Hello World ! </body></pre>	HTML colors can be defined in a name and applied to either the background or the text. Use color property to change the color of your text and bgcolor property to change the color of the background.
HTML fonts tag	<pre>Hello World !</pre>	The font tag is having three attributes called size, color, and face to customize your fonts.
<a> tag	<pre><ahref="https://www.google.com"> Search Engine</pre>	The <a> tag defines a hyperlink. The href attribute specifies the URL of the page
 tag	<pre></pre>	The tag is used to embed an image in an HTML page. The src attribute specifies the path of the image to be displayed. Width and height attributes specifies the width and height of the image in pixels. The alt attribute for the tag specifies an alternate text for an image.
Image Hyperlink	<pre> </pre>	Use an image as hyperlink. We just need to use an image inside hyperlink at the place of text.
HTML ordered List	<pre><ol type="1"> Coffee Tea Milk </pre>	A List is a record of short piece of information. Ordered List or Numbered List (ol) : An ordered list is used when sequence of list items is important.
HTML unordered List	<pre><ul type="square"> Coffee Tea</pre>	Unordered List or Bulleted List (ul) : An unordered list is a collection of related items that

	<pre>Milk </pre>	have no special order or sequence.
HTML Table Tag	<pre><table border= "2" > <tr><th>Item</th> <th>Quantity</th> </tr> <tr> <td>Apple</td> <td>2</td> </tr> <tr> <td>Mango</td> <td>2</td> </tr> <tr> <td>Orange</td> <td>1</td> </tr> </table></pre>	We can create a table to display data in tabular form, using <table> element, with the help of <tr>, <td>, and <th> elements. • In Each table, table row is defined by <tr> tag, table header is defined by <th>, and table data is defined by <td> tags.
HTML Table colspan and rowspan attribute	<pre><table border="1"> <tr> <th>S No</th> <th>Name</th> <th>Class</th> </tr> <tr> <td>1</td> <td>abc</td> <td rowspan="2">9th</td> </tr> <tr> <td>2</td> <td>jkl</td> </tr> <tr> <td>3</td> <td>pqr</td> <td rowspan="2">7th</td> </tr> <tr> <td>4</td> <td>xyz</td> </tr> <tr> <td colspan="2">Total</td> <td>4</td> </tr> </table></pre>	If you want to make a cell span more than one column, you can use the colspan attribute. • If you want to make a cell span more than one row, you can use the rowspan attribute.

HTML Forms and controls

- An HTML form is a section of a document which contains controls such as text fields, password fields, checkboxes, radio buttons, submit button, menus etc.
- An HTML form facilitates the user to enter data that is to be sent to the server for processing such as name, email address, password, phone number, etc.
- HTML forms are required if you want to collect some data from the site visitor.

HTML Form Syntax

```
<form action="server url" method="get/post">
//input controls e.g. textfield, textarea, radiobutton, button
</form>
```

SET A

1. Design a HTML form for Bus Ticket Reservation consisting of fields for Name, Address, contact no, source station(Dropdown list), Destination station, Date of booking, date of journey, no of passenger, name of passenger, gender of passenger etc. (Use proper alignment)
2. Design an HTML form for customer registration visiting a departmental store. Form should consists of fields such as name, contact no., gender, preferred days of purchasing, favorite item(to be selected from a list of items), suggestion etc.. You should provide button to submit as well as reset the form contents.
3. Design an HTML form to take the information of a customer for booking a travel plan consisting of fields such as name, address, contact no., gender, preferred season(Checkboxes), location type(to be selected from a list) etc. You should provide button to submit as well as reset the form contents. (All the fields should be properly aligned)

SET B

1. Design a webpage for an **Employee Profile**, where the second column contains the employee's:

- Employee ID
- Name
- Designation
- Department
- Email
- Photo

In the first column, provide hyperlinks for **Home**, **About Us**, and **Contact HR**.

- Extend the above question:
If the user clicks on the **Contact HR** link, display the HR department's email and phone number.
- Create a form to accept new employee details, including:
 - Name, Employee ID, Designation, Department, Email, and Phone Number.

2. Design a webpage for a **Product Catalog**, where the second column lists product details:

- Product Name
- Product ID
- Price
- Description
- Image of the product

In the first column, include hyperlinks for **Home**, **Contact Us**, and **Categories**.

- Extend the above question:
Clicking on **Categories** should display a list of product categories.
- Create a form to add a new product with the following fields:
 - Product Name, Product ID, Category (dropdown), Price, Description, and Upload Product Image.

3. • Design a webpage for a **School Event**, where the second column contains:

- Event Name

- Date
- Time
- Venue
- Description of the event

In the first column, provide hyperlinks for **Home**, **Event Schedule**, and **Contact Organizer**.

Extend the above question:

If the user clicks on **Event Schedule**, display a table of all upcoming events with dates and times.

- Create a form to register for an event with fields for: Name, Contact Number, Email, Event Name (dropdown), and a message field.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Assignment 2

Introduction to PHP

This assignment extends the previous HTML-based project by introducing PHP, which allows you to add dynamic behavior to the web pages. PHP (Hypertext Preprocessor) is a popular server-side scripting language used to create dynamic websites. The assignment involves two

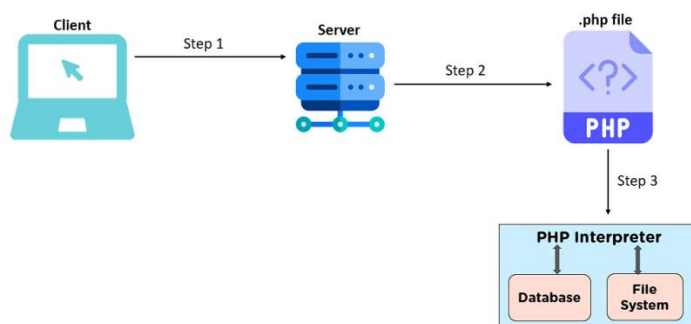
1. Displaying the mark sheet for a student, including details such as name, class, marks, total, and percentage.
2. Creating a login form that authenticates users by matching a username and password, displaying appropriate messages based on the result.

What is PHP

PHP is an open-source, interpreted, and object-oriented scripting language that can be executed at the server-side. PHP is well suited for web development. Therefore, it is used to develop web applications (an application that executes on the server and generates the dynamic page.).

PHP was created by **Rasmus Lerdorf in 1994** but appeared in the market in 1995. **PHP 7.4.0** is the latest version of PHP, which was released on **28 November**.

- PHP stands for Hypertext Preprocessor.
- PHP is an interpreted language, i.e., there is no need for compilation.
- PHP is faster than other scripting languages, for example, ASP and JSP.
- PHP is a server-side scripting language, which is used to manage the dynamic content of the website.
- PHP can be embedded into HTML.
- PHP is an object-oriented language.
- PHP is an open-source scripting language.
- PHP is simple and easy to learn language.



PHP Example

```
<?php
    echo "Hello World!";
?>
```

Output:

Hello, World!

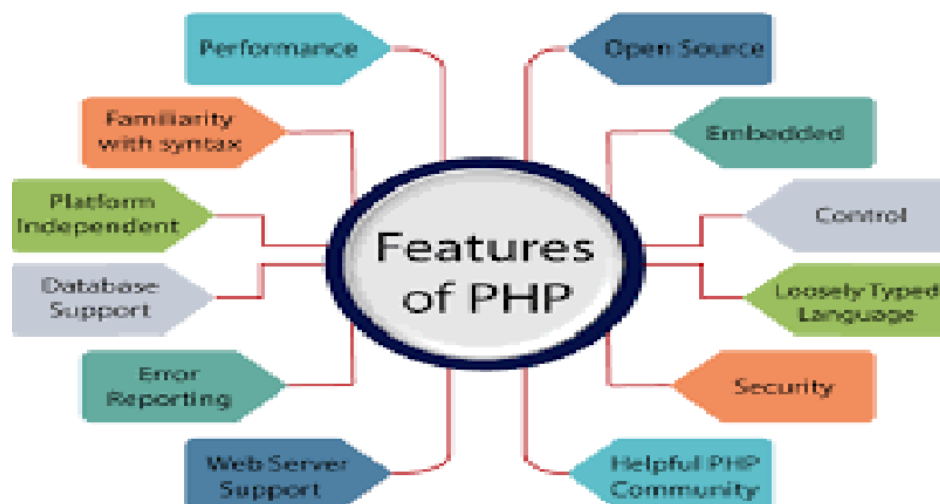
Why use PHP

PHP is a server-side scripting language, which is used to design the dynamic web applications with MySQL database.

- It handles dynamic content, database as well as session tracking for the website.
- You can create sessions in PHP.
- It can access cookies variable and also set cookies.
- It helps to encrypt the data and apply validation.
- PHP supports several protocols such as HTTP, POP3, SNMP, LDAP, IMAP, and many more.
- Using PHP language, you can control the user to access some pages of your website.
- As PHP is easy to install and set up, this is the main reason why PHP is the best language to learn.
- PHP can handle the forms, such as - collect the data from users using forms, save it into the database, and return useful information to the user. **For example** - Registration form.

PHP Features

PHP is very popular language because of its simplicity and open source. There are some important features of PHP given below:



Example of Php program:

```
<?php
// Example PHP program: Greeting based on time

// Get the current hour
```



```

$currentHour = date("H");

// Determine the greeting based on the time
function getGreeting($hour) {
    if ($hour < 12) {
        return "Good morning!";
    } elseif ($hour < 18) {
        return "Good afternoon!";
    } else {
        return "Good evening!";
    }
}

// Display the greeting
echo "<h1>" . getGreeting($currentHour) . "</h1>";

// Display a simple loop
echo "<h2>Numbers from 1 to 5:</h2>";
for ($i = 1; $i <= 5; $i++) {
    echo $i . "<br>";
}

// Closing message
echo "<p>Thank you for using this PHP program!</p>";
?>

```

Explanation:

1. **date ("H"):** Retrieves the current hour in a 24-hour format.
2. **Function:** getGreeting () takes the hour as input and returns an appropriate greeting.
3. **for loop:** Prints numbers from 1 to 5.
4. **echo:** Outputs HTML and dynamic content.

To run this program:

1. Save it as a .php file (e.g., example.php).
2. Place it in the htdocs directory (for XAMPP) or www directory (for WAMP/LAMP servers).
3. Access it in your browser (e.g., <http://localhost/example.php>).

SET A

1. Write a PHP script to accept username and password. If in the first three chances, username and password entered is correct then display second form with “Welcome message” otherwise display error message. [Use Session]
2. Write a PHP script to accept Employee details (Eno, Ename, Address) on first page. On second page accept earning (Basic, DA, HRA). On third page print Employee information (Eno, Ename, Address, Basic, DA, HRA, Total) [Use Session]
3. Write a PHP script to change the preferences of your web page like font style, font size, font color, background color using cookie. Display selected setting on next web page and actual implementation (with new settings) on third page.

SET B

1. Create a form to accept customer information (Name, Addr, MobNo). Once the customer information is accepted, accept product information in the next form (ProdName, Qty,Rate). Generate the bill for the customer in the next form. Bill should contain the customer information and the information of the products entered.

2.Create a PHP program to accept a student's **name**, **class**, and marks of 5 subjects. Display a mark sheet containing the following:

- Student's Name
- Class
- Marks for Each Subject
- Total Marks
- Percentage
- Grade (e.g., A, B, C, D, or Fail)

3. Create a login form with fields for **username** and **password**. Validate the user as follows:

- If both the username and password are "admin", display a "Welcome, Admin!" message.
- If the username is correct but the password is wrong, display "Invalid Password!"
- If the username is incorrect, display "Invalid Username!"

SET C

1. Create a PHP program where the user can select items from a product list (e.g., books, pens, or bags) and input quantities. Display:

- The **selected items** with their unit price.
- The **total quantity** of items.
- The **total cost**.

2. Create a feedback form with the following fields:

- **Name**
- **Email**
- **Feedback message**
- **A rating (1–5 stars)**

Upon form submission, display the user's name, email, and feedback along with a dynamically generated star rating (e.g., if the rating is 4, display 4 stars).

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Assignment 3

Introduction to XML

In this assignment, you are tasked with creating XML files using scripts. XML (Extensible Markup Language) is a flexible, structured format for storing and transporting data. It is designed to be both human-readable and machine-readable. XML is widely used for data exchange between different systems, as it allows complex data to be stored in a hierarchical and structured format.

What is XML?

- XML stands for Extensible Markup Language
- XML is a markup language much like HTML
- XML was designed to store and transport data
- XML was designed to be self-descriptive
- XML is a W3C Recommendation

The Difference Between XML and HTML

- XML was designed to carry data - with focus on what data is
- HTML was designed to display data - with focus on how data looks
- XML tags are not predefined like HTML tags are

XML Does Not Use Predefined Tags

The tags in the example above (like <to> and <from>) are not defined in any XML standard. These tags are "invented" by the author of the XML document.

HTML works with predefined tags like <p>, <h1>, <table>, etc.

With XML, the author must define both the tags and the document structure.

XML Simplifies Things

- XML simplifies data sharing
- XML simplifies data transport
- XML simplifies platform changes
- XML simplifies data availability

Many computer systems contain data in incompatible formats. Exchanging data between incompatible systems (or upgraded systems) is a time-consuming task for web developers. Large amounts of data must be converted, and incompatible data is often lost.

XML stores data in plain text format. This provides a software- and hardware-independent way of storing, transporting, and sharing data.

XML also makes it easier to expand or upgrade to new operating systems, new applications, or new browsers, without losing data.

With XML, data can be available to all kinds of "reading machines" like people, computers, voice machines, news feeds, etc.

SET A

Create an XML File (“Subject.xml”)

Objective:

You need to write a script that creates an XML file named “Subject.xml” to store information about different subjects. This file should contain at least 3 records with details such as subject ID, subject name, class, semester, etc.

Steps to Implement:

1. Structure of the XML file:

- o The XML file will contain multiple <subject> elements, each representing a subject.
- o Each <subject> element will contain attributes and child elements like subject ID, name, class, and semester.

2. Elements and Attributes:

- o Each subject should have a unique ID (subject_id).
- o Subject name, class, and semester should be represented as child elements.

3. XML Format:

- o XML is hierarchical, meaning data is stored in a parent-child structure. Elements are used to define data.
- o The root element should be <subjects>, and each subject will be represented as a <subject> element.

Example of “Subject.xml”:

```
<?xml version="1.0" encoding="UTF-8"?>
<subjects>
<subject subject_id="101">
<name>Mathematics</name>
<class>12th</class>
<semester>1</semester>
</subject>
<subject subject_id="102">
```

```

<name>Physics</name>
<class>12th</class>
<semester>1</semester>
</subject>
<subject subject_id="103">
<name>Chemistry</name>
<class>12th</class>
<semester>1</semester>
</subject>
</subjects>

```

In this XML example:

- ☐ The root element is <subjects>.
- ☐ Each <subject> element contains child elements: <name>, <class>, and <semester>, and an attribute subject_id.
- ☐ Each subject is identified by a unique subject_id.

Steps to Create the XML File in PHP:

You can use PHP to generate and write the XML file. Here's how you can do it:

PHP Script to Create "Subject.xml":

```

<?php
// Create a new DOMDocument instance
$dom = new DOMDocument('1.0', 'UTF-8');
// Create the root element
$subjects = $dom->createElement('subjects');
$dom->appendChild($subjects);
// Creating subject records
$subject1 = $dom->createElement('subject');
$subject1->setAttribute('subject_id', '101');
$subjects->appendChild($subject1);
$subject1->appendChild($dom->createElement('name', 'Mathematics'));
$subject1->appendChild($dom->createElement('class', '12th'));
$subject1->appendChild($dom->createElement('semester', '1'));
$subject2 = $dom->createElement('subject');
$subject2->setAttribute('subject_id', '102');
$subjects->appendChild($subject2);

$subject2->appendChild($dom->createElement('name', 'Physics'));

```

```

$subject2->appendChild($dom->createElement('class', '12th'));
$subject2->appendChild($dom->createElement('semester', '1'));
$subject3 = $dom->createElement('subject');
$subject3->setAttribute('subject_id', '103');
$subjects->appendChild($subject3);
$subject3->appendChild($dom->createElement('name', 'Chemistry'));
$subject3->appendChild($dom->createElement('class', '12th'));
$subject3->appendChild($dom->createElement('semester', '1'));
// Save the XML file
$dom->formatOutput = true; // Makes the XML file pretty
$dom->save('Subject.xml');
?>

```

In this PHP script:

- ☐ We use the DOMDocument class to create and manipulate XML documents.
- ☐ We create a root element <subjects>.
- ☐ For each subject, a <subject> element is created with an attribute subject_id and child elements (name, class, and semester).
- ☐ The XML content is saved to a file named Subject.xml.

Set A

1. Write a script that generates an XML file named "Student.xml" containing at least 5 student records. Each record should include the following data:

- Student ID
- Student Name
- Age
- Gender
- Program

2. Write a script to create an XML file named "Books.xml" containing at least 4 records for books in a library. Each record should include:

- Book ID
- Title
- Author
- Genre
- Publication Year

3. Write a script to generate an XML file called "Enrollment.xml" containing at least 3 records. Each record should have:

- Enrollment ID
- Student ID
- Course Name
- Semester
- Year

4. Write a script to create an XML file called "Employees.xml" containing records for 5 employees. Each record should contain:

- Employee ID
- Name
- Position
- Department
- Date of Joining

5. Write a script that generates an XML file named "Courses.xml" with at least 3 records. Each course should have:

- Course ID
- Course Name
- Credits
- Department
- Instructor

6. Write a script to create an XML file called "Products.xml" containing 4 product records. Each record should include:

- Product ID
- Product Name
- Category
- Price
- Stock Quantity

7. Write a script to generate an XML file called "CourseRegistrations.xml" that stores 5 records of students registering for courses. Each record should contain:

- Registration ID
- Student Name
- Course Name
- Instructor
- Date of Registration

8. Write a script to create an XML file called "Timetable.xml" containing 3 records for a weekly class timetable. Each record should include:

- Day of the Week
- Class Name
- Instructor
- Time Slot
- Classroom

1) Write a XML program which shows how you can easily read and display the contents of an XML document using SimpleXML.

3) Write a PHP Script to read book.XML and print book details in tabular format using simple XML. (Content of book.XML are (bookcode , bookname , author , year , price)

Set B

1. Write a PHP script to create XML file named "Course.xml" Store the details of 5 students who are in SYBBACA. 2) Write PHP script to generate an XML code in the following format Bob Dylan USA Columbia 10.90 1985 Save the file with name "CD.xml". 3) Write PHP script to generate an XML code in the following format Signature of the Instructor 20 Sanguinaria canadensis 4 Mostly Shady \$2.44 031599 Save the file with name "plant.xml".

2. Write a script to create "cricket.xml" file with multiple elements as shown below: Write a script to add multiple elements in "cricket.xml" file of category, country="Australia"

3. Write a script to create "breakfast.xml" file with multiple elements as shown below: French Fries Rs45

Young youths are very much interested to eat it 650 Write a script to add multiple elements in “breakfast.xml” file of category, Juice

4. Create a XML file which gives details of movies available in “Mayanagari CD Store” from following categories a) Classical b) Action c) Horror Elements in each category are in the following format -
--- ---- Save the file with name “movies.xml”

4. Create an application that reads “book.xml” file into simple XML object. Display attributes and elements (Hint:simple_xml_load_file() function).

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

ASSIGNMENT 4

Databases (SQL Injection)

SQL Injection is a type of security vulnerability that occurs when an attacker is able to manipulate a website's SQL queries by injecting arbitrary SQL code into the input fields. It is one of the most critical security risks for web applications, as it can lead to unauthorized access to sensitive data, data manipulation, or even full control over the database.

In this assignment, you're tasked with demonstrating two types of SQL Injection attacks:

1. Login SQL Injection Attack: Attempting to log into another user's account without knowing the correct password.
2. UPDATE Statement SQL Injection Attack: Modifying another user's profile data by injecting malicious SQL code into an UPDATE query.

SQL Injection

SQL injection is a code injection technique that might destroy your database.

SQL injection is one of the most common web hacking techniques.

SQL injection is the placement of malicious code in SQL statements, via web page input.

SQL in Web Pages

SQL injection usually occurs when you ask a user for input, like their username/userid, and instead of a name/id, the user gives you an SQL statement that you will **unknowingly** run on your database.

Look at the following example which creates a **SELECT** statement by adding a variable (txtUserId) to a select string. The variable is fetched from user input (getRequestString):

Example:

```
txtUserId = getRequestString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

SET A

1. Login Bypass Using SQL Injection

Objective: Demonstrate how SQL injection can be used to bypass login authentication and gain unauthorized access.

Scenario:

You are given access to a web application with a login form that takes a username and password. The backend database has the following table:

```
CREATE TABLE users (
```

```
    id INT PRIMARY KEY,  
    username VARCHAR(255) UNIQUE,  
    password VARCHAR(255),
```

role VARCHAR(50)
);

Tasks:

1. Analyze the login functionality and attempt to log in as an admin user without knowing the password.
2. Identify a vulnerable SQL query used to validate user credentials. (Assume it looks something like this: `SELECT * FROM users WHERE username='$username' AND password='$password';`)
3. Test different payloads in the username or password field, such as:
 - username: admin' --
 - password: ' OR '1'='1

Deliverables:

- Write down the SQL query that the server executes after injection.
- Provide evidence of successfully logging in (e.g., a screenshot of the admin dashboard or session details).
- Explain why the injection worked.

2. Unauthorized Profile Modification Using SQL Injection

Objective: Perform SQL injection to modify another user's profile data through an UPDATE query.

Scenario:

The application has a "Profile Update" feature allowing users to update their name and email. Assume the backend query looks like this:

```
UPDATE users SET name='$name', email='$email' WHERE id=$user_id;
```

Tasks:

1. Identify a vulnerability in the input fields for name or email.
2. Inject a payload to update another user's profile information, such as changing their email to attacker@example.com. For example:
 - Input in the "Name" field: John'; UPDATE users SET email='attacker@example.com' WHERE username='victim' --
3. Verify if the unauthorized modification was successful by viewing the updated profile.

Deliverables:

- Provide the payload used to exploit the vulnerability.
- Show the modified profile details as evidence.
- Explain how the injection bypassed the intended access control

3. Insert New Admin User Using SQL Injection

Objective: Use SQL injection to insert a new admin user into the database.

Scenario:

There is a registration form with fields for username and password. The backend query to insert new users looks like this:

```
INSERT INTO users (username, password, role) VALUES ('$username', '$password', 'user');
```

Tasks:

1. Craft an injection payload to add a new user with admin privileges, such as:
 - Username: attacker'); INSERT INTO users (username, password, role) VALUES ('new_admin', 'admin_pass', 'admin') --
2. Verify that the new admin user has been added successfully by logging in.

Deliverables:

- Show the SQL query generated after injection.
- Provide evidence of successful admin account creation.

4. SQL Injection to Retrieve Sensitive Data

Objective: Extract sensitive user data using SQL injection.

Scenario:

The application has a search feature where users can search for usernames. The backend query looks like this:

```
SELECT username, email FROM users WHERE username LIKE '%$search%';
```

Tasks:

1. Inject a payload to retrieve all usernames and email addresses, such as:
 - Input in the search field: %' UNION SELECT username, email FROM users --
2. Display the retrieved data to confirm the injection.

Deliverables:

- Provide the payload used for the attack.
- Show the sensitive data extracted from the database.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

ASSIGNMENT 5

Authorization and Authentication

Authentication

Definition:

Authentication is the process of verifying the identity of a user or system. It ensures that the person or entity attempting to access a system is who they claim to be.

Key Points:

- It is the first step in a security process.
- Focuses on validating credentials such as:
 - **Username and Password**
 - **Biometric Data** (fingerprint, facial recognition)
 - **Security Tokens** (OTP, smart cards)
- Authentication answers the question: *"Who are you?"*

Types of Authentication:

1. **Single-Factor Authentication (SFA):**
 - Uses one method (e.g., password) to verify identity.
2. **Two-Factor Authentication (2FA):**
 - Combines two different methods, like a password and a one-time code sent to a mobile device.
3. **Multi-Factor Authentication (MFA):**
 - Uses multiple methods such as password, biometric data, and a security token for enhanced security.

Examples of Authentication:

- Logging into a website with a username and password.
- Scanning a fingerprint to unlock a smartphone.
- Entering an OTP received on email or SMS.

2. Authorization

Definition:

Authorization is the process of granting or denying access to resources based on the permissions or privileges assigned to a user.

Key Points:

- Comes after authentication.
- Determines what actions a user is allowed to perform or what data they can access.
- Authorization answers the question: *"What are you allowed to do?"*

Types of Authorization:

1. **Role-Based Access Control (RBAC):**
 - Access is granted based on the roles assigned to users (e.g., Admin, Editor, Viewer).

2. **Attribute-Based Access Control (ABAC):**
 - Access is granted based on attributes like user location, time of access, or type of device.
3. **Policy-Based Access Control (PBAC):**
 - Uses predefined policies and conditions to manage access.

Examples of Authorization:

- A regular user can view their account details but cannot modify other users' information.
- Admin users can add or delete data, while regular users can only read data.
- A student can access their own grades but not the grades of other students.

SET A-

Assignment 1: Password Strength Validation

Objective: Design an application to validate the strength of a password.

Scenario:

Create a program that accepts a username and password. The password must meet the following criteria:

1. Contain at least one number.
2. Contain at least one uppercase and one lowercase letter.
3. Contain at least one special symbol (e.g., @, #, \$).
4. Have a minimum length of 8 characters.

Tasks:

1. Prompt the user to enter a username and password.
2. Validate the password based on the above criteria.
3. Display the following messages:
 - If the password is strong: "You entered a strong password."
 - If the password fails the criteria: "Enter a complex password."

Deliverables:

- Source code for the application.
- Screenshots or examples of outputs for both valid and invalid passwords.

Assignment 2: User Registration with Password Encryption

Objective: Design an application to register a user and securely store their information.

Scenario:

Create a program that accepts the following details from a user:

- Full Name
- Contact Number
- Email ID

- Password
- Confirm Password

The application should:

1. Ensure that the password and confirm password fields match.
2. Encrypt the password using a hashing algorithm (e.g., bcrypt, SHA-256).
3. Store the user details, including the encrypted password, in a database.

Tasks:

1. Accept and validate the inputs.
2. If validation is successful, encrypt the password and save the details in a database table structured as follows:

```
sql
CopyEdit
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(255),
    contact_number VARCHAR(15),
    email_id VARCHAR(255) UNIQUE,
    password_hash TEXT
);
```

3. Display a success message upon successful registration. If validation fails, show the appropriate error message.

Deliverables:

- Source code for the application.
- Screenshot of the database table with stored data.
- Examples of inputs for both successful and failed registrations.

Assignment 3: Login System with Encrypted Password Verification

Objective: Implement a login system that verifies user credentials against encrypted passwords stored in a database.

Scenario:

Use the database table created in Assignment 2. Create a login application where:

1. Users input their email ID and password.
2. The application retrieves the stored hashed password from the database.
3. It compares the entered password (after hashing) with the stored hashed password.

Tasks:

1. Prompt the user for their email ID and password.
2. Validate the credentials by comparing the hashed password.
3. Display:

- "Login successful" if the credentials are correct.
- "Invalid credentials" if the login fails.

Deliverables:

- Source code for the login system.
- Example of a successful login and a failed login attempt.

Assignment 4: Password Reset System

Objective: Design a system that allows users to reset their password securely.

Scenario:

Extend the previous assignments to include a password reset feature. The process involves:

1. Users entering their email ID to request a reset.
2. Generating a temporary token or OTP (One-Time Password) and sending it to the user.
3. Users entering the OTP and a new password.
4. Validating the OTP and updating the database with the new encrypted password.

Tasks:

1. Create a mechanism to generate and store the OTP temporarily (e.g., in memory or a database).
2. Accept the OTP, new password, and confirm password from the user.
3. Validate the OTP and password, and update the database with the new encrypted password.

Deliverables:

- Source code for the password reset system.
- Example of the reset process, including OTP validation and database update.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

Assignment 6

Web Services

Web services are crucial components in modern web development, as they enable communication between different software applications over the internet. This assignment focuses on creating a program to access an online weather API and a simple web service to manage dairy product prices.

Web services are software applications that enable communication and data exchange between different systems over a network (usually the internet). They use standardized protocols and technologies to allow interoperability between heterogeneous systems.

➤ Web Services

Web services are designed to support machine-to-machine interaction. They provide a platform-independent way of communication, enabling different applications built on different technologies to work together.

➤ Key Features:

- **Platform Independence:** Applications written in different programming languages can communicate.
- **Interoperability:** Enables interaction between heterogeneous systems.
- **Standardized Protocols:** Uses protocols like HTTP, XML, JSON, SOAP, and REST.

➤ Types of Web Services

1. SOAP Web Services:

- Uses **Simple Object Access Protocol (SOAP)**, a protocol that relies on XML.
- Provides a standardized way to communicate.
- Example: A banking system transferring data between servers.
- **Key Components:**
 - **WSDL (Web Services Description Language):** Describes the service and its methods.
 - **SOAP Messages:** XML-based messages exchanged between client and server.

2. RESTful Web Services:

- Based on the **Representational State Transfer (REST)** architecture.
- Uses standard HTTP methods (GET, POST, PUT, DELETE).
- Example: Retrieving weather data from a REST API.
- **Key Characteristics:**
 - Lightweight and faster compared to SOAP.
 - Uses JSON or XML for data exchange.
 - Stateless communication.

3. Core Components of Web Services

1. Service Provider:

- The server hosting the web service (e.g., API server).
- 2. **Service Consumer:**
 - The client application consuming the web service (e.g., web application, mobile app).
- 3. **Service Registry (Optional):**
 - A directory for discovering web services. For SOAP, this is often **UDDI** (Universal Description, Discovery, and Integration).

4. Common Protocols and Standards

1. **HTTP/HTTPS:** The transport protocol for communication. HTTPS ensures secure communication.
2. **XML/JSON:** Data formats used for exchanging information.
3. **WSDL (SOAP-specific):** Defines the structure of the SOAP web service.
4. **REST Standards:** Follows a resource-based architecture using URIs.

5. Practical Applications of Web Services

1. **Payment Gateways:** Services like PayPal provide APIs for payment processing.
2. **Social Media Integration:** APIs for posting updates or retrieving user data.
3. **Weather Forecasting:** REST APIs provide real-time weather data.
4. **E-commerce:** Product catalogs and order management systems expose APIs for third-party integrations.

6. Practical Implementation Steps

SOAP Web Service:

1. **Define the Service:**
 - Use WSDL to define available operations and data types.
2. **Develop the Service:**
 - Use a programming language like Java or .NET to create the SOAP server.
3. **Consume the Service:**
 - Use client tools or libraries to call the SOAP API (e.g., Postman, SOAP UI).

RESTful Web Service:

1. **Define Resources:**
 - Identify the resources (e.g., users, products) to expose via the service.
2. **Design Endpoints:**
 - Use meaningful URLs:

```
sql
CopyEdit
GET /users → Retrieve all users
POST /users → Create a new user
```

Develop the API:

- Use frameworks like Flask (Python), Spring Boot (Java), or Express.js (Node.js).

3. Test the API:

- Use tools like Postman or cURL to test HTTP requests.

➤ Tools and Technologies

- **For REST API Development:**

- Python: Flask, Django REST Framework
- Java: Spring Boot
- JavaScript: Express.js

- **For SOAP API Development:**

- Java: Apache CXF, JAX-WS
- .NET: WCF (Windows Communication Foundation)

- **Testing Tools:**

- Postman, SOAP UI, cURL

- **Security Add-ons:**

- OAuth2, API keys, JWT (JSON Web Tokens).

➤ Advantages of Web Services

- Cross-platform compatibility.
- Reusable components.
- Reduced application integration time.
- Supports distributed systems

SET A

1. Write a program to access online weather API.

2. Write a simple web service called Dairy Product Price Service. This keeps a Dairy Product Price table to record the different kinds of product prices. It will supply 3 services to the client so that clients can add product price, delete product price, update product price

3. Write a program that fetches weather data from an online API (e.g., OpenWeatherMap or WeatherAPI). The program should:

- Accept a city name as input.
- Display the current weather, temperature, and humidity.
- Provide a 5-day weather forecast.
- Handle errors such as invalid city names or API connection issues.

4. Create a simple web service called Library Book Service. It should manage a table of books and their details (title, author, genre, and price). The service must provide the following functionalities:

- **Add Book:** Add a new book to the table.
- **Delete Book:** Remove a book based on its ID.
- **Update Book:** Modify the details of an existing book.
- **Get Book Details:** Retrieve details of all books or a specific book by ID.

SET B

1. Write a program that uses a currency exchange API (e.g., ExchangeRate-API or Fixer.io) to perform the following:

- Convert a specified amount from one currency to another.
- Display the exchange rate for a given currency pair.
- Handle invalid currency codes or API errors gracefully.

2. Develop a web service called `Student Management Service`. This service should manage student records and offer:

- **Add Student:** Add a new student record (name, roll number, class, etc.).
- **Delete Student:** Delete a record using the roll number.
- **Update Student:** Update details of a specific student.
- **Get Student Details:** Fetch details of all students or one by roll number.

3. Write a program that accesses an online movie database API (e.g., TMDb or OMDb API) to:

- Search for movies based on keywords or genres.
- Display movie details such as title, release date, and rating.
- Recommend similar movies based on user preferences or the selected movie.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Need Improvement []

4: Complete []

5: Well Done []

Signature of Instructor

ASSIGNMENT 7

Introduction to OWASP (Open Web Application Security Project)

OWASP (Open Web Application Security Project) is a non-profit organization focused on improving the security of software. OWASP provides freely available security-related resources, including guidelines, tools, and best practices, to help organizations protect their applications from vulnerabilities.

One of the most important contributions from OWASP is the OWASP Top 10, which highlights the top security risks that web applications and APIs face. In this assignment, we will explore the OWASP Top 10 for Web Applications and the OWASP Top 10 for APIs, which are crucial for understanding and mitigating security risks in web development.

OWASP

The **Open Web Application Security Project (OWASP)** is a nonprofit foundation focused on improving the security of software. It provides free tools, resources, and knowledge to help organizations, developers, and security professionals secure web applications against potential threats and vulnerabilities.

Objectives of OWASP:

1. Raise awareness about web application security risks.
2. Provide open-source tools and methodologies for secure software development.
3. Share resources, best practices, and documentation to promote secure coding practices.
4. Encourage collaboration between developers, testers, and security professionals.

Key Components of OWASP

1. OWASP Top 10:

The OWASP Top 10 is a flagship project that lists the most critical security risks to web applications. It is updated periodically based on data collected from security professionals worldwide.

Some examples of risks include:

- Injection (e.g., SQL Injection)
- Broken Authentication
- Cross-Site Scripting (XSS)
- Insecure Direct Object References

2. OWASP Tools and Resources:

- **ZAP (Zed Attack Proxy):** A free tool for identifying vulnerabilities in web applications.
- **Dependency-Check:** A tool to identify known vulnerabilities in project dependencies.
- **Cheat Sheets:** Guides that cover best practices for secure development.

3. OWASP Security Knowledge Framework (SKF):

A knowledge base for developers to understand security principles and practices for building secure applications.

4. OWASP Projects:

OWASP hosts several open-source projects, including documentation, code, and tools for different aspects of application security.

Importants of OWASP

- **Promotes Security Awareness:** Helps developers and organizations understand common threats and vulnerabilities.
- **Encourages Best Practices:** Guides developers in adopting secure development life cycle (SDLC) processes.
- **Community-Driven:** Contributions from a global community of security experts ensure the latest and most reliable information.
- **Open Source:** All OWASP tools and resources are free, making security accessible to everyone.

Applications of OWASP in Security Practices

1. **Web Application Testing:** Using OWASP ZAP to identify vulnerabilities during development and testing phases.
2. **Secure Coding Practices:** Following OWASP cheat sheets to avoid security loopholes during coding.
3. **Vulnerability Management:** Identifying and mitigating vulnerabilities using OWASP Dependency-Check.
4. **Compliance:** Many security standards and frameworks (e.g., PCI DSS) reference OWASP resources for web application security.

SET A

- 1.Explain OWASP Top 10 threats for web introduction.
- 2.Explain OWASP Top 10 threats for API introduction
- 3.Exaplain real-world examples of at least **three threats** (e.g., SQL Injection, Cross-Site Scripting, and Security Misconfiguration).
4. Write a short report explaining:
 - a) The identified threat.
 - b) How the threat was exploited.
 - c) The steps you took to mitigate the vulnerability.
5. Explain Postman or OWASP ZAP to test the API for security vulnerabilities.
Generate a report summarizing the issues found and fixed.

Assignment Evaluation

0: Not Done [] 1: Incomplete [] 2: Late Complete [] 3: Needs Improvement []
4: Complete [] 5: Well Done []

Signature of the Instructor

ASSIGNMENT 8

Introduction to Web Security Audit Tools

➤ Introduction to Web Security Audit Tools

Web security audit tools are specialized software programs used to identify vulnerabilities, misconfigurations, and security gaps in web applications, servers, and APIs. These tools play a crucial role in ensuring the integrity, confidentiality, and availability of online systems by simulating potential attacks, detecting security weaknesses, and providing actionable insights for remediation.

➤ Web Security Audit Tools

Web security audit tools automate the process of testing a web application's defenses against various security threats. These tools are essential for:

- **Identifying vulnerabilities** like SQL Injection, Cross-Site Scripting (XSS), and Broken Authentication.
- **Assessing compliance** with security standards and regulations such as OWASP, PCI DSS, and GDPR.
- **Improving security posture** by providing recommendations to mitigate risks.

➤ Objectives of Web Security Audit Tools

1. **Detection of Vulnerabilities:** Identify security issues in web applications, APIs, and infrastructure.
2. **Prevention of Attacks:** Help secure systems against cyberattacks such as data breaches, DoS attacks, and malware.
3. **Compliance Enforcement:** Ensure that web applications adhere to security best practices and standards.
4. **Continuous Monitoring:** Facilitate regular checks to maintain the security of evolving web systems.

➤ Types of Vulnerabilities Detected

1. **Injection Attacks:** SQL, LDAP, and command injections.
2. **Cross-Site Scripting (XSS):** Unauthorized script execution in a user's browser.
3. **Broken Authentication and Session Management:** Weak login systems or improperly managed user sessions.
4. **Security Misconfigurations:** Exposed sensitive files, weak permissions, and outdated software.
5. **Insecure APIs:** Excessive data exposure, lack of rate limiting, and weak authorization mechanisms.

➤ Common Features of Web Security Audit Tools

1. **Automated Vulnerability Scanning:** Detect common security flaws with minimal manual effort.
2. **Detailed Reporting:** Provide a summary of identified vulnerabilities with severity levels and remediation steps.
3. **Simulation of Real Attacks:** Emulate real-world attacks to test the application's resilience.
4. **Integration with Development Pipelines:** Enable security checks during development using CI/CD tools.
5. **Compliance Checks:** Evaluate applications against industry standards and guidelines.

➤ Popular Web Security Audit Tools

1. **OWASP ZAP (Zed Attack Proxy):**
 - Open-source tool for web application security testing.
 - Features include automated scans, a proxy for manual testing, and passive vulnerability detection.
2. **Burp Suite:**
 - Comprehensive tool used for manual and automated security testing.
 - Features include vulnerability scanning, request modification, and advanced testing extensions.
3. **Acunetix:**

- Automated vulnerability scanner for web applications.
 - Detects vulnerabilities like SQL Injection, XSS, and misconfigurations.
4. **Nikto:**
 - Open-source web server scanner.
 - Identifies outdated server software, insecure files, and improper configurations.
 5. **Nessus:**
 - A widely-used vulnerability assessment tool for web servers and networks.
 - Provides detailed vulnerability analysis and compliance checks.

➤ **Steps in Conducting a Web Security Audit**

1. **Define the Scope:**
Identify the assets to be audited, such as web applications, servers, or APIs.
2. **Choose the Right Tool:**
Select an appropriate web security audit tool based on the application's complexity and requirements.
3. **Perform the Audit:**
Run automated scans or manual tests to identify vulnerabilities.
4. **Analyze Results:**
Review the audit reports, prioritize issues based on severity, and prepare a remediation plan.
5. **Fix Vulnerabilities:**
Apply patches, update configurations, and implement secure coding practices.
6. **Re-Test:**
Conduct a follow-up audit to confirm the effectiveness of the fixes.

➤ **Benefits of Using Web Security Audit Tools**

1. **Proactive Risk Management:** Detects vulnerabilities before attackers can exploit them.
2. **Improved Security Posture:** Enhances the overall resilience of web applications.
3. **Cost-Effective:** Reduces the cost of addressing security breaches by preventing them.
4. **Regulatory Compliance:** Ensures adherence to industry regulations and standards.
5. **Time-Saving:** Automates the auditing process, saving time compared to manual testing.

➤ **Advantages of Burp Suite:**

Comprehensive Security Testing: Burp Suite provides a wide range of security tools, from intercepting proxies to automated vulnerability scanners, making it a one-stop solution for web application testing.

Extensibility: Burp Suite supports extensions, allowing users to add functionality and integrate with other tools or services.

Active Community and Support: Burp Suite has a large user community and provides professional support for enterprise users.

Powerful Reporting: Burp Suite generates detailed, customizable reports that can be shared with clients or stakeholders.

Limitations of Burp Suite:

Cost: While the Community Edition is free, the Professional Edition requires a paid

license, which can be costly for individual users or small teams.

Complexity: Burp Suite has a steeper learning curve compared to other tools like OWASP ZAP, and it may take time to become proficient in using all of its features.

Resource Intensive: Burp Suite can be resource-intensive, especially when scanning large web applications, which may slow down the process on lower-end systems.

Example of Burp Suite Usage:

1. Intercepting Traffic: Set up Burp Suite as a proxy and configure your browser to route traffic through it. As you navigate through the application, Burp Suite will intercept and log the requests.

2. Active Scan: Use Burp Suite's automated scanner to detect common vulnerabilities such as SQL injection or XSS. The tool will send payloads to the application and analyze the responses to identify vulnerabilities.

3. Manual Testing with Repeater: You can use Burp Suite's Repeater tool to manually modify and send HTTP requests to test for additional vulnerabilities or to analyze specific endpoints more thoroughly.

SET A

- 1.Explain OWASP ZAP scanner .
- 2.Explain Burp Suite Scanner
- 3.Explain the purpose of OWASP ZAP (Zed Attack Proxy) in web application security testing.

SET B

1. Describe its key features, such as automated scans, passive scanning, active scanning, and the proxy server functionality.
2. Explain the role of Burp Suite in web application penetration testing.
3. Explain key components, such as the proxy, scanner, repeater, intruder, and comparer tools.

Assignment Evaluation

0: Not Done [] 1: Incomplete [] 2: Late Complete [] 3: Needs Improvement []
4: Complete [] 5: Well Done []

Signature of the Instructor

Date of Completion ____ / ____ / ____